UNIVERSIDAD MARIANO GÁLVEZ
CENTRO UNIVERSITARIO AMATITLÁN
FACULTAD DE INGENIERÍA EN SISTEMAS DE INFORMACIÓN

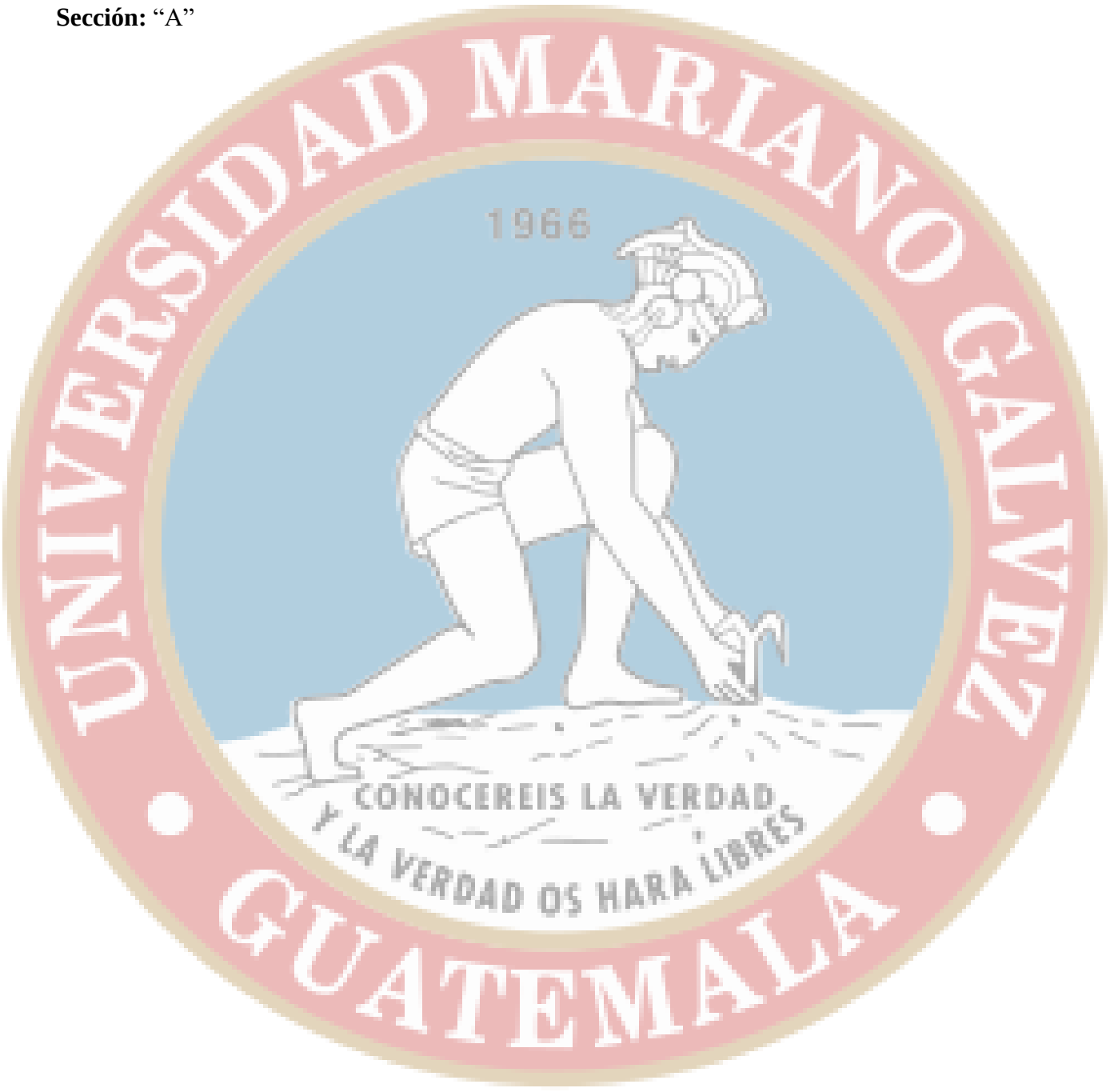
Curso:

Catedrático:

Estudiante: Brandon Elí Hernández Morales

Carné: 6691-26-15969

Sección: “A”



Librerías usadas para la interfaz gráfica

import javafx.application.Application; // nuestra base de la aplicación, es para desplegar la interfaz en una ventana que podamos controlar

import javafx.stage.Stage; // la ventana principal de la aplicación (minimizar, cerrar, etc.)

import javafx.scene.Scene; // esta es la escena principal que contiene todo lo que ve el usuario

import javafx.collections.FXCollections; // crea listas que sirven para los menús desplegables, como inicio y destino

import javafx.scene.control.Button; // botón de la interfaz, como el de "Calcular ruta"

import javafx.scene.control.ComboBox; // menú desplegable para elegir el edificio de inicio y destino

import javafx.scene.control.Label; // etiquetas de texto simples, para títulos

import javafx.scene.control.TextArea; // caja de texto donde se muestra el desglose del cálculo de Dijkstra

import javafx.scene.layout.BorderPane; // divide la ventana en zonas

import javafx.scene.layout.HBox; // acomoda elementos en fila

import javafx.scene.layout.VBox; // acomoda elementos en columna

Librerías usadas para representarlo de forma 2d

import javafx.scene.canvas.Canvas; // lienzo 2D donde dibujamos el mapa plano

import javafx.scene.canvas.GraphicsContext; // herramientas para dibujar líneas, texto y rectángulos

import javafx.scene.text.Font; // fuente de letra para los textos

import javafx.scene.text.FontWeight; // grosor de la letra

import javafx.scene.text.TextAlignment; // alineación del texto dentro del Canvas

Librerías usadas para representarlo de forma 3d

import javafx.geometry.Point3D; // representa vectores con coordenadas X, Y, Z

import javafx.scene.AmbientLight; // crea iluminación para visualizar correctamente los colores

import javafx.scene.Group; // agrupa los diferentes objetos gráficos

import javafx.scene.PerspectiveCamera; // cámara que permite ver el mundo 3D

import javafx.scene.SceneAntialiasing; // suaviza los bordes

import javafx.scene.SubScene; // espacio dentro de la interfaz donde vive el mundo 3D

import javafx.scene.paint.Color; // colores de los elementos gráficos

import javafx.scene.paint.PhongMaterial; // material/color que se aplica a las formas 3D

import javafx.scene.shape.Box; // cajas 3D usadas para edificios, piso, cancha y parqueo

import javafx.scene.shape.Cylinder; // usado para caminos y elementos de la cancha

import javafx.scene.text.Text; // nombres que aparecen dentro del mundo 3D

import javafx.scene.transform.Rotate; // permite realizar rotaciones en el espacio 3D

Librería usada para crear la estructura con la que se hace Djikstra

import java.util.*; // Map, List, PriorityQueue, Set, etc.

Programación orientada a objetos

Usamos la POO para crear clases y objetos y así agrupar y organizar los elementos del programa. Por ejemplo, tenemos la clase Edificio para guardar los datos de cada edificio, la clase Conexion para guardar los caminos y sus distancias, y Posicion2D para guardar las posiciones del mapa. Esto nos sirve para mantener los datos ordenados y trabajar

con ellos de una forma más sencilla

```
static class Edificio {
    String nombre;
    double x, z, alto;
    TipoLugar tipo;
    Edificio(String nombre, double x, double z, double alto, TipoLugar tipo) {
        this.nombre = nombre; this.x = x; this.z = z; this.alto = alto; this.tipo = tipo;
    }
}

static class Conexion {
    String origen, destino;
    int distancia;
    Conexion(String origen, String destino, int distancia) {
        this.origen = origen; this.destino = destino; this.distancia = distancia;
    }
}

static class Posicion2D {
    double x, y;
    Posicion2D(double x, double y) { this.x = x; this.y = y; }
}
```

Declaración de variables

En esta parte declaramos las variables que vamos a utilizar durante el programa. Tenemos variables para guardar los edificios, conexiones, posiciones del mapa, colores, objetos 3D, la ruta calculada, la cámara y elementos de la interfaz. Por ejemplo, en este caso, Private es también POO por que agrupa los datos que se usarán en la representación 3d

```
32  public class RutaMinimaCampus3D extends Application {
58      private static final double PISO_Y = -5;
59
60      private static final String COLOR_FONDO = "#121212";
61      private static final String COLOR_PANEL = "#1c1c1f";
62      private static final String COLOR_ACENTO = "#1DB975";
63      private static final String COLOR_TEXTO = "#e8e8e8";
64
65      private static final Color COLOR_EDIFICIO = Color.web("#585b63");
66      private static final Color COLOR_CANCHA = Color.web("#2f6b3c");
67      private static final Color COLOR_PARQUEO = Color.web("#3a3a3a");
68      private static final Color COLOR_EDIFICIO_RUTA = Color.web(COLOR_ACENTO);
69      private static final Color COLOR_CAMINO = Color.web("#787c86");
70      private static final Color COLOR_CAMINO_RUTA = Color.web(COLOR_ACENTO);
71
72      private static final double CAJA_2D_ANCHO = 90;
73      private static final double CAJA_2D_ALTO = 34;
74
75      private final Map<String, Edificio> edificios = new LinkedHashMap<>();
76      private final List<Conexion> conexiones = new ArrayList<>();
77      private final Map<String, List<String[]>> adyacencia = new HashMap<>();
78      private final Map<String, Posicion2D> posiciones2D = new LinkedHashMap<>();
79
80      private final Map<String, Box> cajasPorEdificio = new HashMap<>();
81      private final Map<String, Color> colorNormalPorEdificio = new HashMap<>();
82      private final Map<Conexion, Cylinder> cilindrosPorConexion = new LinkedHashMap<>();
83
84      private List<String> rutaResaltada = new ArrayList<>();
85
86      private final Rotate rotateX = new Rotate(-25, Rotate.X_AXIS);
87      private final Rotate rotateY = new Rotate(-35, Rotate.Y_AXIS);
88      private double anclaX, anclaY, anguloAnclaX, anguloAnclaY;
89      private PerspectiveCamera camera;
90
91      private Label labelResultado;
92      private Canvas canvas2D;
```

Asignar un lugar en la hoja o plano 2d, como es 2d solo se usa x y y

```
212
213     private void inicializarPosiciones2D() {
214         posiciones2D.put(key: "Biblioteca", new Posicion2D(x: 12, y: 20));
215         posiciones2D.put(key: "Rectoria", new Posicion2D(x: 155, y: 20));
216         posiciones2D.put(key: "Cafeteria", new Posicion2D(x: 290, y: 20));
217         posiciones2D.put(key: "Administracion", new Posicion2D(x: 430, y: 20));
218         posiciones2D.put(key: "Auditorio", new Posicion2D(x: 12, y: 150));
219         posiciones2D.put(key: "Laboratorio", new Posicion2D(x: 150, y: 150));
220         posiciones2D.put(key: "Gimnasio", new Posicion2D(x: 290, y: 150));
221         posiciones2D.put(key: "Enfermeria", new Posicion2D(x: 430, y: 150));
222         posiciones2D.put(key: "Cancha", new Posicion2D(x: 80, y: 280));
223         posiciones2D.put(key: "Parqueo", new Posicion2D(x: 360, y: 280));
224     }
225
226     private void agregarEdificio(String nombre, double x, double z, double alto) {
227         agregarEdificio(nombre, x, z, alto, TipoLugar.EDIFICIO);
228     }
229
230     private void agregarEdificio(String nombre, double x, double z, double alto, TipoLugar tipo) {
231         edificios.put(nombre, new Edificio(nombre, x, z, alto, tipo));
232         adyacencia.put(nombre, new ArrayList<>());
```

Parecido para hacerlo en la versión 3d, pero agregamos z para que haya tres dimensiones

```
private void inicializarDatos() {
    agregarEdificio(nombre: "Biblioteca", -300, -200, alto: 140);
    agregarEdificio(nombre: "Rectoria", -50, -320, alto: 100);
    agregarEdificio(nombre: "Cafeteria", x: 200, -250, alto: 160);
    agregarEdificio(nombre: "Administracion", x: 420, -100, alto: 120);
    agregarEdificio(nombre: "Auditorio", -380, z: 80, alto: 90);
    agregarEdificio(nombre: "Laboratorio", -100, z: 40, alto: 180);
    agregarEdificio(nombre: "Gimnasio", x: 180, z: 60, alto: 110);
    agregarEdificio(nombre: "Enfermeria", x: 400, z: 150, alto: 100);
    agregarEdificio(nombre: "Cancha", -200, z: 280, alto: 40, TipoLugar.CANCHA);
    agregarEdificio(nombre: "Parqueo", x: 100, z: 300, alto: 35, TipoLugar.PARQUEO);
```

Para establecer las conexiones, básicamente las distancias entre un edificio con el otro

```
195
196         agregarConexion(origen: "Biblioteca", destino: "Rectoria", distancia: 120);
197         agregarConexion(origen: "Rectoria", destino: "Cafeteria", distancia: 100);
198         agregarConexion(origen: "Cafeteria", destino: "Administracion", distancia: 90);
199         agregarConexion(origen: "Biblioteca", destino: "Auditorio", distancia: 60);
200         agregarConexion(origen: "Rectoria", destino: "Laboratorio", distancia: 70);
201         agregarConexion(origen: "Cafeteria", destino: "Gimnasio", distancia: 65);
202         agregarConexion(origen: "Administracion", destino: "Enfermeria", distancia: 55);
203         agregarConexion(origen: "Auditorio", destino: "Laboratorio", distancia: 80);
204         agregarConexion(origen: "Laboratorio", destino: "Gimnasio", distancia: 75);
205         agregarConexion(origen: "Gimnasio", destino: "Enfermeria", distancia: 60);
206         agregarConexion(origen: "Auditorio", destino: "Cancha", distancia: 90);
207         agregarConexion(origen: "Laboratorio", destino: "Cancha", distancia: 100);
208         agregarConexion(origen: "Gimnasio", destino: "Parqueo", distancia: 85);
209         agregarConexion(origen: "Enfermeria", destino: "Parqueo", distancia: 70);
210         agregarConexion(origen: "Cancha", destino: "Parqueo", distancia: 120);
211     }
```

Para crear los elementos del plano 2d, edificios, rutas, botones, etc

```
434     private VBox crearControles() {
435         String[] nombres = edificios.keySet().toArray(new String[0]);
436
437         ComboBox<String> comboInicio = new ComboBox<>(FXCollections.observableArrayList(nombres));
438         ComboBox<String> comboFin = new ComboBox<>(FXCollections.observableArrayList(nombres));
439         comboInicio.setValue(nombres[0]);
440         comboFin.setValue(nombres[nombres.length - 1]);
441         comboInicio.setStyle("-fx-background-radius: 8;");
442         comboFin.setStyle("-fx-background-radius: 8;");
443
444         Button botonCalcular = new Button("Calcular ruta");
445         botonCalcular.setStyle(
446             "-fx-background-color: " + COLOR_ACENTO + ";" +
447             "-fx-text-fill: white;" +
448             "-fx-font-weight: bold;" +
449             "-fx-background-radius: 8;" +
450             "-fx-padding: 8 20 8 20;"
451         );
452         botonCalcular.setOnAction(e -> calcularRuta(comboInicio.getValue(), comboFin.getValue()));
453
454         Label lblInicio = new Label("Inicio:");
455         Label lblFin = new Label("Destino:");
456         lblInicio.setStyle("-fx-text-fill: " + COLOR_TEXTO + ";");
457         lblFin.setStyle("-fx-text-fill: " + COLOR_TEXTO + ";");
458
459         labelResultado = new Label("");
460         labelResultado.setStyle("-fx-text-fill: " + COLOR_TEXTO + "; -fx-font-size: 12;");
461
462         HBox fila = new HBox(12, lblInicio, comboInicio, lblFin, comboFin, botonCalcular);
463         fila.setAlignment(javafx.geometry.Pos.CENTER_LEFT);
464         fila.setPadding(new javafx.geometry.Insets(16, 16, 6, 16));
465
466         VBox contenedor = new VBox(4, fila, labelResultado);
467         VBox.setMargin(labelResultado, new javafx.geometry.Insets(0, 16, 12, 16));
468         contenedor.setStyle("-fx-background-color: " + COLOR_PANEL + ";");
```

Para crear toda la parte 3d del programa

```
241
242     private Box crearPiso() {
243         Box piso = new Box(1200, 10, 900);
244         piso.setMaterial(new PhongMaterial(Color.web("□#262b26")));
245         return piso;
246     }
247
248     private Color colorNormalDe(Edificio e) {
249         switch (e.tipo) {
250             case CANCHA: return COLOR_CANCHA;
251             case PARQUEO: return COLOR_PARQUEO;
252             default: return COLOR_EDIFICIO;
253         }
254     }
255
256     private Group crearEdificio(Edificio e) {
257         colorNormalPorEdificio.put(e.nombre, colorNormalDe(e));
258         switch (e.tipo) {
259             case CANCHA: return crearCancha(e);
260             case PARQUEO: return crearParqueo(e);
261             default: return crearEdificioNormal(e);
262         }
263     }
264
265     private Text crearEtiqueta(Edificio e) {
266         Text etiqueta = new Text(e.nombre);
267         etiqueta.setFont(Font.font(14));
268         etiqueta.setFill(Color.WHITE);
269         etiqueta.setTranslateX(e.x - (e.nombre.length() * 3.5));
270         etiqueta.setTranslateZ(e.z);
271         etiqueta.setTranslateY(PISO_Y - e.alto - 12);
272         return etiqueta;
273     }
274
```

```
275     private Group crearEdificioNormal(Edificio e) {
276         Box caja = new Box(60, e.alto, 60);
277         caja.setTranslateX(e.x);
278         caja.setTranslateZ(e.z);
279         caja.setTranslateY(PISO_Y - e.alto / 2.0);
280         caja.setMaterial(new PhongMaterial(colorNormalDe(e)));
281         cajasPorEdificio.put(e.nombre, caja);
282
283         return new Group(caja, crearEtiqueta(e));
284     }
285
286     // Cancha: superficie plana verde con borde, linea central y circulo central,
287     // en vez de una caja alta como los edificios.
288     private Group crearCancha(Edificio e) {
289         double ancho = 130, profundo = 80, grosor = 4;
290
291         Box piso = new Box(ancho, grosor, profundo);
292         piso.setTranslateX(e.x);
293         piso.setTranslateZ(e.z);
294         piso.setTranslateY(PISO_Y - grosor / 2.0);
295         piso.setMaterial(new PhongMaterial(colorNormalDe(e)));
296         cajasPorEdificio.put(e.nombre, piso);
297
298         double yLinea = PISO_Y - grosor - 0.5;
299         double grosorLinea = 2;
300         PhongMaterial blanco = new PhongMaterial(Color.WHITE);
301
302         Box bordeSuperior = new Box(ancho, 1, grosorLinea);
303         bordeSuperior.setMaterial(blanco);
304         bordeSuperior.setTranslateX(e.x);
305         bordeSuperior.setTranslateZ(e.z - profundo / 2.0);
306         bordeSuperior.setTranslateY(yLinea);
307
308         Box bordeInferior = new Box(ancho, 1, grosorLinea);
309         bordeInferior.setMaterial(blanco);
```

```
        bordeInferior.setMaterial(blanco);
        bordeInferior.setTranslateX(e.x);
        bordeInferior.setTranslateZ(e.z + profundo / 2.0);
        bordeInferior.setTranslateY(yLinea);

        Box bordeIzquierdo = new Box(grosorLinea, 1, profundo);
        bordeIzquierdo.setMaterial(blanco);
        bordeIzquierdo.setTranslateX(e.x - ancho / 2.0);
        bordeIzquierdo.setTranslateZ(e.z);
        bordeIzquierdo.setTranslateY(yLinea);

        Box bordeDerecho = new Box(grosorLinea, 1, profundo);
        bordeDerecho.setMaterial(blanco);
        bordeDerecho.setTranslateX(e.x + ancho / 2.0);
        bordeDerecho.setTranslateZ(e.z);
        bordeDerecho.setTranslateY(yLinea);

        Box lineaCentral = new Box(grosorLinea, 1, profundo);
        lineaCentral.setMaterial(blanco);
        lineaCentral.setTranslateX(e.x);
        lineaCentral.setTranslateZ(e.z);
        lineaCentral.setTranslateY(yLinea);

        Cylinder circuloCentral = new Cylinder(14, 1);
        circuloCentral.setMaterial(blanco);
        circuloCentral.setTranslateX(e.x);
        circuloCentral.setTranslateZ(e.z);
        circuloCentral.setTranslateY(yLinea);

        return new Group(piso, bordeSuperior, bordeInferior, bordeIzquierdo, bordeDerecho,
                lineaCentral, circuloCentral, crearEtiqueta(e));
    }

    // Parqueo: superficie plana oscura con lineas blancas de espacios,
    // en vez de una caja alta como los edificios.
    private Group crearParqueo(Edificio e) {
        double ancho = 110, profundo = 70, grosor = 4;

        Box piso = new Box(ancho, grosor, profundo);
        piso.setTranslateX(e.x);
        piso.setTranslateZ(e.z);
        piso.setTranslateY(PISO_Y - grosor / 2.0);
        piso.setMaterial(new PhongMaterial(colorNormalDe(e)));
        cajasPorEdificio.put(e.nombre, piso);

        double yLinea = PISO_Y - grosor - 0.5;
        Group lineas = new Group();
        int numLineas = 4;
        double espaciado = ancho / (numLineas + 1.0);

        for (int i = 1; i <= numLineas; i++) {
            double offsetX = -ancho / 2.0 + espaciado * i;
            Box linea = new Box(1.5, 1, profundo * 0.8);
            linea.setMaterial(new PhongMaterial(Color.web("■#e8e8e8")));
            linea.setTranslateX(e.x + offsetX);
            linea.setTranslateZ(e.z);
            linea.setTranslateY(yLinea);
            lineas.getChildren().add(linea);
        }

        return new Group(piso, lineas, crearEtiqueta(e));
    }

    private Cylinder crearCamino(Edificio origen, Edificio destino) {
        Point3D p1 = new Point3D(origen.x, PISO_Y - 1, origen.z);
        Point3D p2 = new Point3D(destino.x, PISO_Y - 1, destino.z);
```

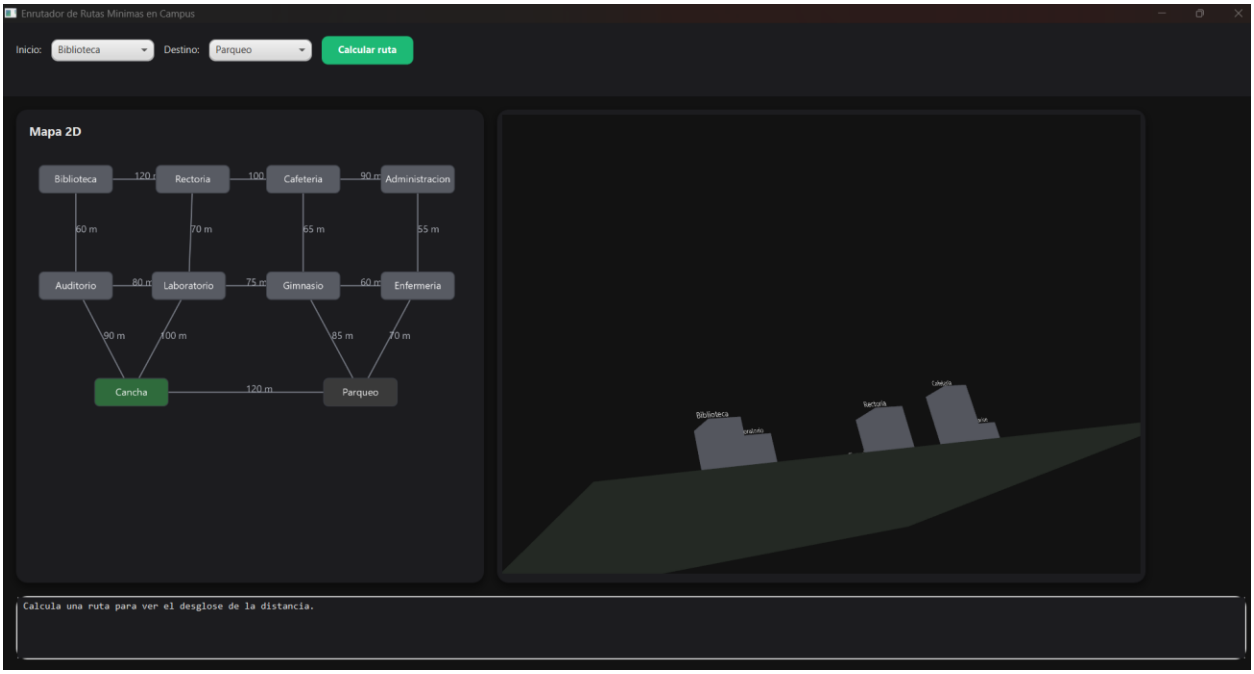
```
375
376         Point3D diferencia = p2.subtract(p1);
377         double longitud = diferencia.magnitude();
378         Point3D ejeY = new Point3D(0, 1, 0);
379         Point3D ejeRotacion = diferencia.crossProduct(ejeY);
380         double angulo = Math.acos(diferencia.normalize().dotProduct(ejeY));
381
382         Cylinder camino = new Cylinder(2.5, longitud);
383         camino.setMaterial(new PhongMaterial(COLOR_CAMINO));
384
385         Point3D medio = p1.midpoint(p2);
386         camino.setTranslateX(medio.getX());
387         camino.setTranslateY(medio.getY());
388         camino.setTranslateZ(medio.getZ());
389         camino.getTransforms().add(new Rotate(-Math.toDegrees(angulo), ejeRotacion));
390
391         return camino;
```

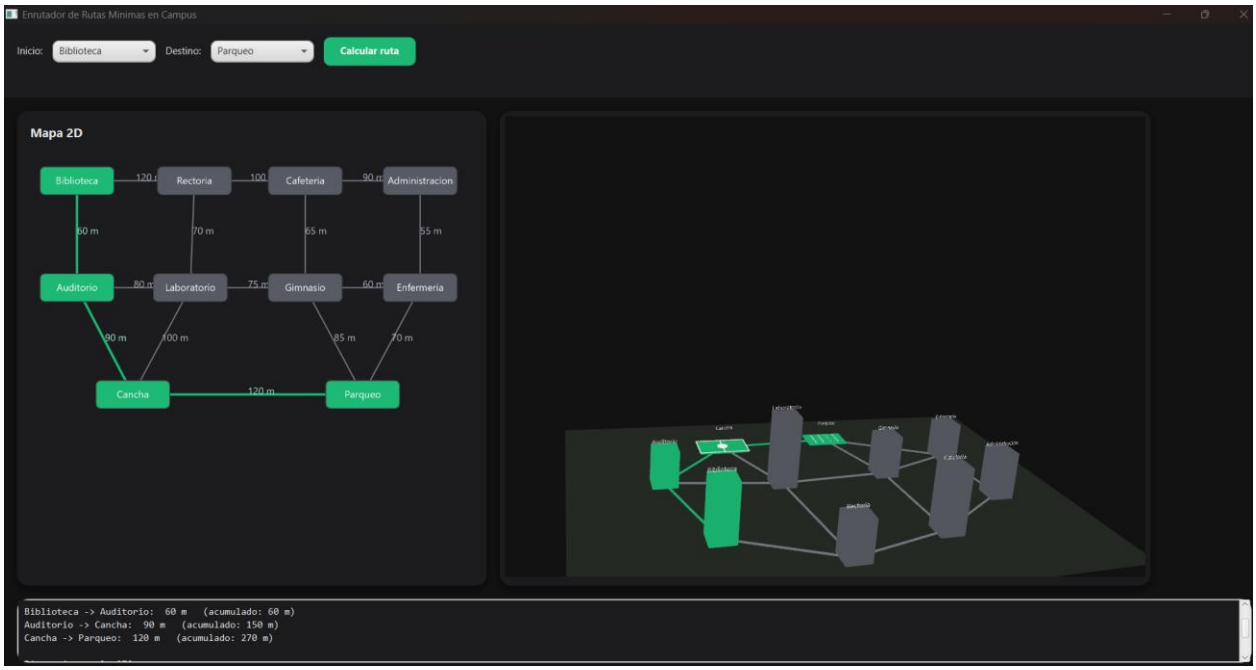
Ciclos usados para crear Djkstra

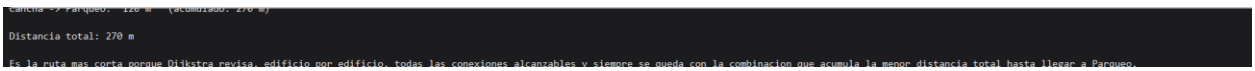
```
472     private void calcularRuta(String inicio, String fin) {
473         if (inicio == null || fin == null || inicio.equals(fin)) {
474             labelResultado.setText("Elige un inicio y un destino distintos.");
475             return;
476         }
477 
478         Map<String, Integer> distancias = new HashMap<>();
479         Map<String, String> previos = new HashMap<>();
480         for (String nombre : edificios.keySet()) distancias.put(nombre, Integer.MAX_VALUE);
481         distancias.put(inicio, value: 0);
482 
483         PriorityQueue<String> pendientes = new PriorityQueue<>(Comparator.comparingInt(distancias::get));
484         pendientes.add(inicio);
485         Set<String> visitados = new HashSet<>();
486 
487         while (!pendientes.isEmpty()) {
488             String actual = pendientes.poll();
489             if (visitados.contains(actual)) continue;
490             visitados.add(actual);
491 
492             for (String[] vecino : adyacencia.get(actual)) {
493                 String nombreVecino = vecino[0];
494                 int peso = Integer.parseInt(vecino[1]);
495                 int nuevaDistancia = distancias.get(actual) + peso;
496                 if (nuevaDistancia < distancias.get(nombreVecino)) {
497                     distancias.put(nombreVecino, nuevaDistancia);
498                     previos.put(nombreVecino, actual);
499                     pendientes.add(nombreVecino);
500                 }
501             }
502         }
```

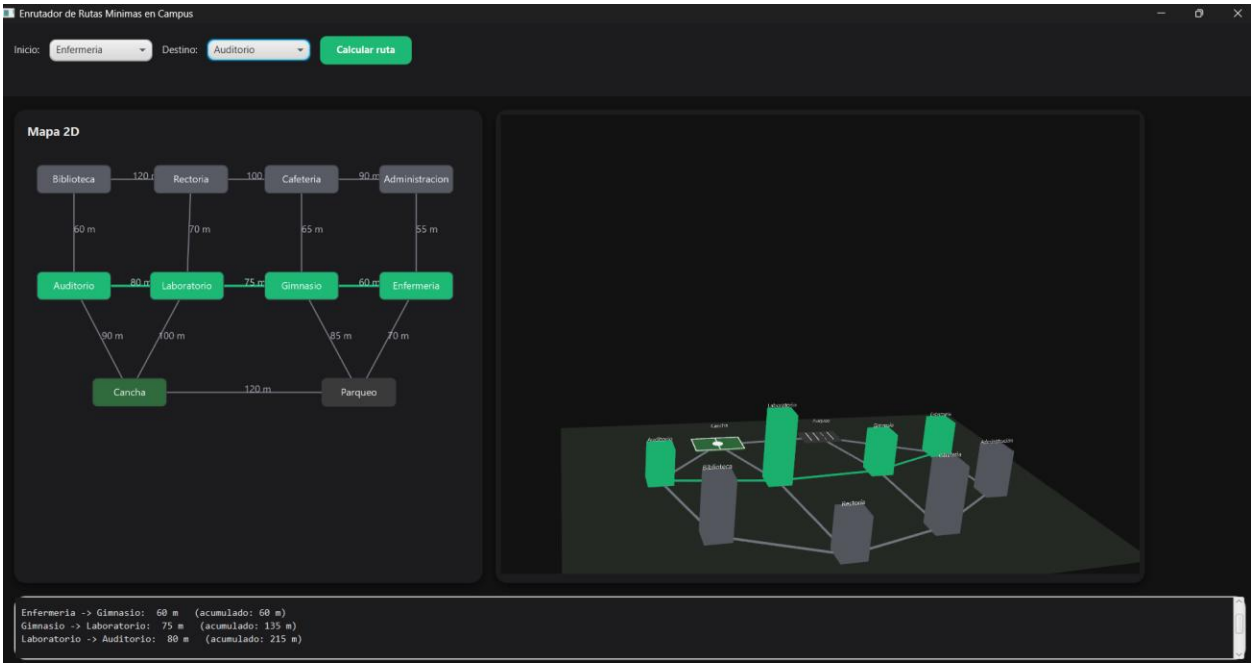
```
503 
504         if (distancias.get(fin) == Integer.MAX_VALUE) {
505             labelResultado.setText("No existe una ruta entre " + inicio + " y " + fin + ".");
506             rutaResaltada = new ArrayList<>();
507             areaCalculo.setText("No existe una ruta entre " + inicio + " y " + fin + ".");
508         } else {
509             List<String> ruta = new ArrayList<>();
510             String paso = fin;
511             while (paso != null) {
512                 ruta.add(index: 0, paso);
513                 paso = previos.get(paso);
514             }
515             rutaResaltada = ruta;
516             labelResultado.setText("");
517             areaCalculo.setText(construirExplicacionCalculo(ruta, inicio, fin));
518         }
519 
520         actualizarColores();
521         dibujar2D();
522     }
523 
```

Aquí usamos los ciclos para realizar la lógica de Dijkstra. El while mantiene el algoritmo funcionando mientras tengamos edificios pendientes por revisar. El for recorre las conexiones o vecinos del edificio actual. Luego el if compara las distancias y, si encuentra una más corta, la actualiza. Así se repite el proceso hasta encontrar la ruta más corta.


Enrutador de Rutas Mínimas en Campus
Inicio:
Biblioteca
Destino:
Parqueo
Calcular ruta
Mapa 2D
Biblioteca
120 m
Rectoria
100 m
Cafeteria
90 m
Administracion
60 m
70 m
65 m
55 m
Auditorio
80 m
Laboratorio
75 m
Gimnasio
60 m
Enfermeria
90 m
100 m
85 m
70 m
Cancha
120 m
Parqueo
Calcula una ruta para ver el desglose de la distancia.


Enrutador de Rutas Mínimas en Campus
Inicio:
Biblioteca
Destino:
Parqueo
Calcular ruta
Mapa 2D
Biblioteca
120 m
Rectoria
100 m
Cafeteria
90 m
Administracion
60 m
70 m
65 m
55 m
Auditorio
80 m
Laboratorio
75 m
Gimnasio
60 m
Enfermeria
90 m
100 m
85 m
70 m
Cancha
120 m
Parqueo
Biblioteca -> Auditorio: 60 m (acumulado: 60 m)
Auditorio -> Cancha: 90 m (acumulado: 150 m)
Cancha -> Parqueo: 120 m (acumulado: 270 m)


Distancia total: 270 m
Es la ruta mas corta porque Dijkstra revisa, edificio por edificio, todas las conexiones alcanzables y siempre se queda con la combinacion que acumula la menor distancia total hasta llegar a Parqueo.


Enrutador de Rutas Minimas en Campus
Inicio:
Enfermeria
Destino:
Auditorio
Calcular ruta
Mapa 2D
Biblioteca
120 m
Rectoria
100 m
Cafeteria
90 m
Administracion
60 m
70 m
65 m
55 m
Auditorio
80 m
Laboratorio
75 m
Gimnasio
60 m
Enfermeria
90 m
100 m
85 m
70 m
Cancha
120 m
Parqueo
Enfermeria -> Gimnasio: 60 m (acumulado: 60 m)
Gimnasio -> Laboratorio: 75 m (acumulado: 135 m)
Laboratorio -> Auditorio: 80 m (acumulado: 215 m)